

WEEK 5 → HIBERNATE (ORM FRAMEWORK)

COMPLETE ROADMAP (MONDAY TO FRIDAY)

WEEK 5 OBJECTIVE

After completing Week 5, students should understand:

- ✓ What is Hibernate
 - ✓ Why Hibernate was introduced
 - ✓ ORM Concept
 - ✓ Hibernate Architecture
 - ✓ Hibernate Configuration
 - ✓ Entity Class
 - ✓ SessionFactory
 - ✓ Session
 - ✓ Transaction
 - ✓ CRUD Operations
 - ✓ Hibernate Annotations
 - ✓ Real Project Flow
 - ✓ Hibernate vs JDBC
-

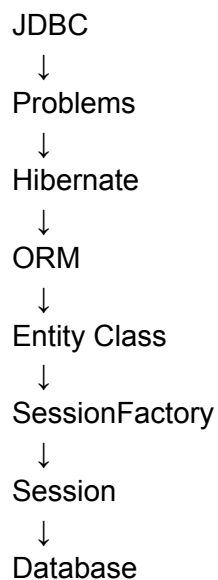


WEEK 5 PLAN

Day	Topic
Day 1	Introduction to Hibernate + ORM
Day 2	Hibernate Architecture + Configuration
Day 3	Entity Class + Annotations
Day 4	Hibernate CRUD Operations
Day 5	Complete Student Registration Project Using Hibernate



WEEK 5 FLOW



DAY 1 → INTRODUCTION TO HIBERNATE + ORM (DEEP)



1. WHAT PROBLEM DID JDBC HAVE?

Till now we used JDBC.

Example:

```
Connection con =  
DriverManager.getConnection();
```

```
PreparedStatement ps =  
con.prepareStatement();
```

```
ResultSet rs =  
ps.executeQuery();
```

For every project we write:

```
Connection  
PreparedStatement  
ResultSet  
SQL Queries
```

again and again.

PROBLEMS IN JDBC

Problem 1

Too Much Code

Example:

```
Connection
```

```
PreparedStatement
```

```
ResultSet
```

```
Close Connection
```

Large code.

Problem 2

SQL Dependency

Need SQL everywhere.

insert into student

update student

delete student

Problem 3

Database Change

Suppose:

MySQL

changes to

Oracle

Many queries need modification.

Problem 4

Object Mapping Problem

Java works with:

Student student

Database works with:

student table

Need conversion.

SOLUTION

Hibernate



2. WHAT IS HIBERNATE?

Hibernate is an ORM Framework.

Used to interact with database using Java Objects instead of SQL.

SIMPLE DEFINITION

Hibernate converts:

Java Object

into

Database Record

automatically.



FLOW

Java Object



Hibernate



SQL Query



Database



REAL EXAMPLE

JDBC

```
insert into student  
values(...)
```

Hibernate

```
session.save(student);
```

Hibernate generates SQL automatically.



3. WHAT IS ORM?

Most Important Interview Question.

ORM =

Object Relational Mapping

BREAKDOWN

Object

```
Student student =  
new Student();
```

Relational

Database Table

Mapping

Connecting Object
with Table



ORM DIAGRAM

Java Object



ORM



Database Table

EXAMPLE

Java

```
Student s =  
new Student();
```

Database

student

table.

Hibernate maps:

Student Object



Student Table



4. WHY ORM IS NEEDED?

Without ORM:

Object



SQL Query



Database

Manual conversion.

With ORM:

Object



ORM



Database

Automatic conversion.



5. REAL WORLD EXAMPLE

Suppose:

Student Object

```
Student s =  
new Student();
```

```
s.setId(1);
```

```
s.setName("Swati");
```

Hibernate converts automatically:

```
insert into student  
(id,name)
```

```
values
```

```
(1,'Swati');
```

Developer writes:

```
session.save(s);
```

Only.

HUGE ADVANTAGE

No SQL required for basic operations.

6. HIBERNATE ARCHITECTURE OVERVIEW

Java Application



Hibernate API



Configuration



SessionFactory



Session



Database

We'll learn each component in upcoming days.

7. JDBC VS HIBERNATE

JDBC	Hibernate
Manual SQL	Automatic SQL
More Code	Less Code
Low Level	High Level
Manual Mapping	Automatic Mapping
Hard Maintenance	Easy Maintenance

EXAMPLE

JDBC

```
PreparedStatement ps =  
con.prepareStatement(  
"insert..."  
);
```

Hibernate

```
session.save(student);
```



8. HIBERNATE ADVANTAGES

Advantage 1

Less Coding

Advantage 2

Automatic SQL Generation

Advantage 3

Database Independent

Advantage 4

ORM Support

Advantage 5

Object-Oriented

Advantage 6

Industry Standard



9. HIBERNATE DISADVANTAGES

Disadvantage 1

Learning Curve

Disadvantage 2

Configuration Required

Disadvantage 3

Can be slower for very complex SQL



10. WHERE IS HIBERNATE USED?

Used in:

- ✓ Banking Applications
 - ✓ E-Commerce Systems
 - ✓ ERP Systems
 - ✓ Hospital Management Systems
 - ✓ Student Management Systems
 - ✓ Spring Boot Projects
-



11. HIBERNATE IN REAL PROJECT

Registration System

Without Hibernate

Servlet



DAO



JDBC



Database

With Hibernate

Servlet



DAO



Hibernate



Database

12. IMPORTANT HIBERNATE TERMINOLOGIES

Term	Meaning
ORM	Object Relational Mapping
Entity	Java Class mapped to Table
SessionFactory	Creates Sessions
Session	Database Operations
Transaction	Commit/Rollback
Configuration	Hibernate Settings

13. INTERVIEW QUESTIONS

What is Hibernate?

ORM Framework for Java.

What is ORM?

Object Relational Mapping.

Why Hibernate?

To reduce JDBC code and automate mapping.

Hibernate works on which concept?

ORM.

Difference between JDBC and Hibernate?

Hibernate automatically generates SQL.



DAY 1 ASSIGNMENT

Theory

1. What is Hibernate?
 2. What is ORM?
 3. Explain Object Relational Mapping.
 4. Difference between JDBC and Hibernate.
 5. Advantages of Hibernate.
-

Practical

Task 1

Create Student class:

```
Student s =  
new Student();
```

Think how Hibernate would convert it into table record.

Task 2

Draw ORM diagram.

```
Object  
↓  
Hibernate  
↓  
Database
```

Task 3

Compare JDBC and Hibernate with 10 points.



WEEK 5 → DAY 2 DEEP

HIBERNATE ARCHITECTURE + CONFIGURATION + SESSIONFACTORY + SESSION + TRANSACTION



DAY 2 OBJECTIVE

By the end of this session students should understand:

- ✓ Hibernate Architecture
 - ✓ Hibernate Components
 - ✓ Maven Dependencies
 - ✓ hibernate.cfg.xml
 - ✓ SessionFactory
 - ✓ Session
 - ✓ Transaction
 - ✓ First Hibernate Project Setup
 - ✓ Internal Working of Hibernate
-



1. RECAP OF DAY 1

We learned:

JDBC



Too Much Code



Hibernate



ORM



Database

JDBC Example

Connection con

PreparedStatement ps

ResultSet rs

Hibernate Example

```
session.save(student);
```

Hibernate automatically generates SQL.



2. HIBERNATE ARCHITECTURE

Most Important Interview Question.

COMPLETE ARCHITECTURE

Java Application



Configuration



SessionFactory



Session



Transaction



Database

UNDERSTAND EACH COMPONENT

Java Application

Contains:

Entity Classes

DAO Classes

Main Method

Example:

Student.java

Configuration

Contains database settings.

Example:

Database URL

Username

Password

Dialect

Stored in:

hibernate.cfg.xml

SessionFactory

Creates Session objects.

Session

Performs database operations.

Transaction

Handles:

Commit
Rollback

Database

MySQL

Oracle

PostgreSQL

etc.



COMPLETE FLOW

Student Object

↓
Configuration
↓
SessionFactory
↓
Session
↓
Transaction
↓
MySQL

3. FIRST HIBERNATE PROJECT STRUCTURE

src/main/java

```
|— entity  
|   Student.java  
|  
|— util  
|   HibernateUtil.java  
|  
|— main  
|   App.java
```

src/main/resources

```
|— hibernate.cfg.xml
```

4. MAVEN DEPENDENCIES

Before Hibernate works,

Maven must download libraries.

pom.xml

```
<dependencies>

<dependency>
<groupId>org.hibernate.orm</groupId>
<artifactId>hibernate-core</artifactId>
<version>6.6.18.Final</version>
</dependency>

<dependency>
<groupId>com.mysql</groupId>
<artifactId>mysql-connector-j</artifactId>
<version>9.3.0</version>
</dependency>

</dependencies>
```

WHY THESE DEPENDENCIES?

Hibernate Core

Provides:

SessionFactory
Session
Transaction
Annotations

MySQL Connector

Provides:

Connection between
Java and MySQL



5. DATABASE CREATION

Create Database

create database hibernatedb;

Use Database

use hibernatedb;

At this stage:

No table creation needed.

Hibernate can create tables automatically.



6. WHAT IS hibernate.cfg.xml?

Most Important File.

Definition

Main configuration file of Hibernate.

Contains database settings.

LOCATION

src/main/resources

hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<hibernate-configuration>

<session-factory>

<property name="hibernate.connection.driver_class">
com.mysql.cj.jdbc.Driver
</property>

<property name="hibernate.connection.url">
jdbc:mysql://localhost:3306/hibernatedb
</property>

<property name="hibernate.connection.username">
root
</property>

<property name="hibernate.connection.password">
root
</property>

<property name="hibernate.dialect">
org.hibernate.dialect.MySQLDialect
</property>

<property name="hibernate.hbm2ddl.auto">
update
</property>

<property name="hibernate.show_sql">
true
</property>

</session-factory>

</hibernate-configuration>
```

7. UNDERSTANDING EACH PROPERTY

Driver Class

com.mysql.cj.jdbc.Driver

Purpose:

Load MySQL Driver.

URL

jdbc:mysql://localhost:3306/hibernatedb

Purpose:

Connect to database.

Username

root

Database username.

Password

root

Database password.

Dialect

MySQLDialect

Tells Hibernate:

Database is MySQL

hbm2ddl.auto

update

Meaning:

Automatically update tables.

show_sql

true

Meaning:

Display SQL Queries
on Console



8. WHAT IS SessionFactory?

Most Important Interview Question.

Definition

SessionFactory creates Session objects.

SIMPLE EXAMPLE

Think:

Water Tank

creates:

Water Bottles

Similarly:

SessionFactory

creates:

Session

FLOW

SessionFactory



Session



Database

CREATE SessionFactory

```
Configuration cfg =  
new Configuration();
```

```
cfg.configure();
```

```
SessionFactory sf =  
cfg.buildSessionFactory();
```

INTERNAL FLOW

hibernate.cfg.xml



Configuration
↓
SessionFactory



9. WHAT IS SESSION?

Most Frequently Used Object.

Definition

Session represents connection between application and database.

Used For

Save

Update

Delete

Fetch

operations.

CREATE SESSION

```
Session session =  
sf.openSession();
```

FLOW

SessionFactory



openSession()



Session



10. WHAT IS TRANSACTION?

Important Concept.

Definition

Transaction manages database changes.

Example

Insert Student.

Without Commit:

Data not saved permanently.

FLOW

Start Transaction



Insert Data



Commit

CREATE TRANSACTION

```
Transaction tx =  
session.beginTransaction();
```

SAVE DATA

```
session.persist(student);
```

COMMIT

```
tx.commit();
```



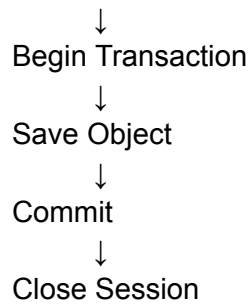
COMPLETE FLOW

```
Session  
↓  
Begin Transaction  
↓  
Save Data  
↓  
Commit  
↓  
Database
```

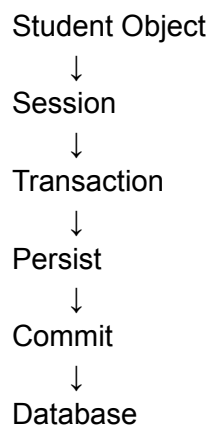


11. FIRST HIBERNATE PROGRAM FLOW

```
Create Configuration  
↓  
Create SessionFactory  
↓  
Open Session
```

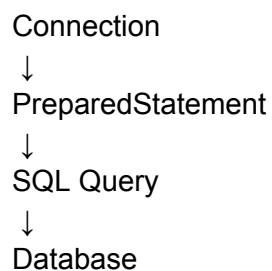


VISUAL DIAGRAM

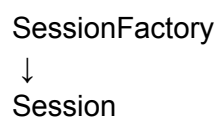


12. HIBERNATE VS JDBC FLOW

JDBC



Hibernate



↓
Transaction
↓
Database

13. COMMON INTERVIEW QUESTIONS

What is SessionFactory?

Creates Session objects.

What is Session?

Performs database operations.

What is Transaction?

Used for Commit and Rollback.

What is hibernate.cfg.xml?

Configuration file of Hibernate.

What is Dialect?

Specifies database type.

What is hbm2ddl.auto=update?

Automatically updates table structure.

Difference between SessionFactory and Session?

SessionFactory	Session
Creates Sessions	Performs DB Operations
One per Application	Many per Application
Heavy Object	Lightweight Object

COMMON STUDENT MISTAKES

Mistake 1

Forgetting:

```
tx.commit();
```

Result:

Data not saved.

Mistake 2

Wrong database URL.

```
jdbc:mysql://localhost:3306/wrongdb
```

Connection fails.

Mistake 3

Wrong username/password.

Mistake 4

Not adding Hibernate dependency.



DAY 2 ASSIGNMENT

Theory

1. Explain Hibernate Architecture.
 2. What is SessionFactory?
 3. What is Session?
 4. What is Transaction?
 5. Explain hibernate.cfg.xml.
 6. Explain all Hibernate properties.
 7. Difference between SessionFactory and Session.
-

Practical

Task 1

Create Maven Project:

HibernateDemo

Task 2

Add Hibernate + MySQL dependencies.

Task 3

Create:

hibernatedb

database.

Task 4

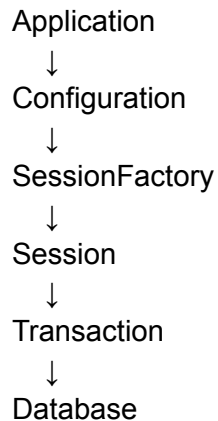
Create:

hibernate.cfg.xml

and configure database.

Task 5

Draw Architecture Diagram:



WEEK 5 → DAY 3 DEEP

ENTITY CLASS + HIBERNATE ANNOTATIONS + TABLE CREATION



DAY 3 OBJECTIVE

By the end of this session students should understand:

- ✓ What is Entity Class
- ✓ Why Entity Class is needed
- ✓ @Entity

- ✓ @Table
 - ✓ @Id
 - ✓ @GeneratedValue
 - ✓ @Column
 - ✓ Hibernate Table Creation
 - ✓ Mapping Java Class to Database Table
 - ✓ First Hibernate Project
-

1. RECAP OF DAY 2

We learned:

Application
↓
Configuration
↓
SessionFactory
↓
Session
↓
Transaction
↓
Database

Today we will learn:

Java Class
↓
Entity Class
↓
Hibernate
↓
Database Table



2. WHAT IS AN ENTITY CLASS?

Most Important Hibernate Concept.

Definition

An Entity Class is a Java class that is mapped to a database table.

SIMPLE DEFINITION

Entity Class

=

Database Table Representation

EXAMPLE

Database Table

student

id	name	course
1	Swati	Java

Equivalent Java Class

```
public class Student {
```

```
    private int id;
```

```
    private String name;
```

```
    private String course;
```

}

Hibernate connects:

Student Class



Student Table



ENTITY MAPPING

Java Class



Hibernate



Database Table



3. WHY ENTITY CLASS IS NEEDED?

Without Entity:

PreparedStatement

SQL Query

ResultSet

With Entity:

Student s =
new Student();

Hibernate automatically converts object into table row.



4. FIRST ENTITY CLASS

Student.java

```
public class Student {  
  
    private int id;  
  
    private String name;  
  
    private String course;  
  
}
```

Question:

How does Hibernate know this is a table?

Answer:

Annotations



5. WHAT ARE ANNOTATIONS?

Annotation = Metadata.

Provides information to Hibernate.

Example:

@Entity

Means:

This class is an Entity.



6. @Entity

Most Important Annotation.

Purpose

Marks class as Entity Class.

Example

```
import jakarta.persistence.Entity;
```

```
@Entity
```

```
public class Student {
```

```
}
```

Meaning:

Student Class



Database Table

WITHOUT @Entity

Hibernate ignores class.

No table created.



7. @Table

Purpose:

Specify table name.

Example

@Entity

@Table(name="student")

```
public class Student {  
  
}
```

Meaning:

Class Name



Student

Table Name



student

WHY USE @Table?

Suppose:

StudentInformation

class name.

But database table should be:

student

Then:

@Table(name="student")



8. @Id

Most Important Annotation.

Purpose:

Defines Primary Key.

Database:

id int primary key

Java:

@Id

private int id;

Meaning:

id field



Primary Key



PRIMARY KEY REVISION

Primary Key:

- ✓ Unique
- ✓ Cannot be Duplicate
- ✓ Identifies Record

Example

id name

1 Swati

2 Rahul

No duplicate IDs allowed.

9. @GeneratedValue

Purpose:

Auto-generate ID.

Without GeneratedValue

```
student.setId(1);
```

must be written manually.

With GeneratedValue

Hibernate generates ID automatically.

Example

@Id

@GeneratedValue

```
private int id;
```

Database Output

id	name
1	Swati
2	Rahul
3	Amit

IDs generated automatically.



GeneratedValue Strategies

AUTO

```
@GeneratedValue(  
strategy=GenerationType.AUTO  
)
```

Hibernate decides.

IDENTITY

```
@GeneratedValue(  
strategy=GenerationType.IDENTITY  
)
```

Uses MySQL auto_increment.

Most common.

Example

@Id

```
@GeneratedValue(  
strategy=GenerationType.IDENTITY  
)
```

```
private int id;
```



10. @Column

Purpose:

Map field to database column.

Example

```
@Column(name="student_name")
```

```
private String name;
```

Meaning:

Java Field



name

Database Column



student_name

Example

Database

id	student_name
----	--------------

1	Swati
---	-------

Java

```
private String name;
```

Mapped using:

```
@Column(name="student_name")
```



COMPLETE ENTITY CLASS

```
package entity;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
@Table(name="student")
```

```
public class Student {
```

```
    @Id
```

```
    @GeneratedValue(  
        strategy=GenerationType.IDENTITY  
    )
```

```
    private int id;
```

```
    @Column(name="student_name")
```

```
    private String name;
```

```
    @Column(name="student_course")
```

```
    private String course;
```

```
    public Student() {
```

```
    }
```

```
    public int getId() {
```

```
return id;
}

public void setId(int id) {
this.id = id;
}

public String getName() {
return name;
}

public void setName(String name) {
this.name = name;
}

public String getCourse() {
return course;
}

public void setCourse(String course) {
this.course = course;
}
}
```

11. REGISTER ENTITY IN hibernate.cfg.xml

Hibernate must know Entity exists.

Add:

```
<mapping
class="entity.Student"/>
```

Complete:

```
<session-factory>
```

...

```
<mapping  
class="entity.Student"/>
```

```
</session-factory>
```

12. HOW TABLE CREATION HAPPENS?

Suppose:

```
@Entity
```

```
@Table(name="student")
```

exists.

And:

```
hibernate.hbm2ddl.auto=  
update
```

When project runs:

Hibernate checks:

student table exists?

If NO

↓

Creates automatically.



FLOW

Student Entity
↓
Hibernate Starts
↓
Checks Table
↓
Creates Table
↓
MySQL



GENERATED TABLE

Hibernate creates:

```
create table student(  
  
id int auto_increment,  
  
student_name varchar(255),  
  
student_course varchar(255),  
  
primary key(id)  
  
);
```



13. COMPLETE PROJECT STRUCTURE

src/main/java

```
|— entity  
|   Student.java  
|  
|— util  
|   HibernateUtil.java  
|
```

|— main
| App.java

src/main/resources

|— hibernate.cfg.xml



14. REAL WORLD FLOW

Student Object

```
Student s =  
new Student();
```

```
s.setName("Swati");
```

```
s.setCourse("Java");
```

Hibernate:

Student Object



Entity Mapping



SQL Query



Database Table

Database

id	student_nam e	student_cours e
1	Swati	Java



15. INTERVIEW QUESTIONS

What is Entity Class?

Java class mapped to database table.

What is @Entity?

Marks class as Hibernate Entity.

What is @Table?

Specifies table name.

What is @Id?

Defines Primary Key.

What is @GeneratedValue?

Auto-generates primary key.

What is @Column?

Maps Java field to database column.

What happens if @Entity is missing?

Hibernate ignores class.

COMMON STUDENT MISTAKES

Mistake 1

No default constructor.

Wrong:

```
public Student(String name){  
  
}
```

Only.

Correct:

```
public Student(){  
  
}
```

Mistake 2

Missing:

@Id

Hibernate error.

Mistake 3

Forgetting:

```
<mapping  
class="entity.Student"/>
```

Table not created.

Mistake 4

Wrong package name.



DAY 3 ASSIGNMENT

Theory

1. What is Entity Class?
 2. Why Entity Class is needed?
 3. Explain @Entity.
 4. Explain @Table.
 5. Explain @Id.
 6. Explain @GeneratedValue.
 7. Explain @Column.
 8. How Hibernate creates table automatically?
-

Practical

Task 1

Create:

Student.java

Entity Class.

Task 2

Add:

@Entity

@Table

@Id

@GeneratedValue

@Column

Task 3

Create fields:

id

name

course
fees

Task 4

Run project.

Verify table creation in MySQL.

Task 5

Change:

```
private String city;
```

Add new field.

Run project again.

Observe:

New column automatically added.



WEEK 5 → DAY 4 DEEP

HIBERNATE CRUD OPERATIONS (CREATE, READ, UPDATE, DELETE)



DAY 4 OBJECTIVE

By the end of this session students should understand:

- ✓ What is CRUD

- ✓ Insert Data using Hibernate
 - ✓ Fetch Data using Hibernate
 - ✓ Update Data using Hibernate
 - ✓ Delete Data using Hibernate
 - ✓ Transaction Management
 - ✓ Session Methods
 - ✓ Real Student Management Example
 - ✓ Complete Flow from Java Object to Database
-

1. WHAT IS CRUD?

CRUD is the most common operation in every application.

Full Form

C → Create
R → Read
U → Update
D → Delete

Real Example

Student Management System

Operation	Example
Create	Add Student
Read	View Student
Update	Update Student Course

Delete

Remove Student



CRUD FLOW

Student Object



Hibernate



Database



2. PROJECT STRUCTURE

src/main/java

```
|— entity
|   Student.java
|
|— util
|   HibernateUtil.java
|
|— main
|   App.java
```

src/main/resources

```
|— hibernate.cfg.xml
```



3. STUDENT ENTITY

@Entity

@Table(name="student")

public class Student {

@Id

@GeneratedValue(

```
strategy=GenerationType.IDENTITY
)

private int id;

private String name;

private String course;

private double fees;

public Student() {

}

public Student(
String name,
String course,
double fees
){
this.name=name;
this.course=course;
this.fees=fees;
}

}
```



4. HIBERNATE UTIL CLASS

Purpose:

Create SessionFactory once.

```
package util;

import org.hibernate.*;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {

private static SessionFactory sessionFactory;

static {
```

```
Configuration cfg =  
new Configuration();  
  
cfg.configure();  
  
sessionFactory =  
cfg.buildSessionFactory();  
  
}  
  
public static SessionFactory  
getSessionFactory(){  
  
return sessionFactory;  
  
}  
  
}
```



5. CREATE OPERATION (INSERT)

Most common operation.

STEP 1

Create Student Object

```
Student s =  
new Student(  
"Swati",  
"Java",  
5000  
);
```

STEP 2

Open Session

```
Session session =  
HibernateUtil  
.getSessionFactory()  
.openSession();
```

STEP 3

Begin Transaction

```
Transaction tx =  
session.beginTransaction();
```

STEP 4

Save Object

Hibernate 6:

```
session.persist(s);
```

STEP 5

Commit

```
tx.commit();
```

STEP 6

Close Session

```
session.close();
```

COMPLETE INSERT PROGRAM

```
Session session =  
HibernateUtil  
.getSessionFactory()  
.openSession();  
  
Transaction tx =  
session.beginTransaction();  
  
Student s =  
new Student(  
"Swati",  
"Java",  
5000  
);  
  
session.persist(s);  
  
tx.commit();  
  
session.close();  
  
System.out.println(  
"Student Saved"  
);
```



INTERNAL FLOW

```
Student Object  
↓  
persist()  
↓  
Hibernate  
↓  
INSERT SQL  
↓  
MySQL
```

GENERATED SQL

Hibernate generates:

```
insert into student  
(name,course,fees)
```

values

```
('Swati','Java',5000);
```

6. READ OPERATION (FETCH SINGLE RECORD)

Database

id	name
1	Swati

Fetch Student.

Hibernate Method

```
session.get()
```

Example

Student student =

```
session.get(
```

```
Student.class,  
1  
);
```

Meaning:

Fetch Student
whose id = 1

COMPLETE PROGRAM

```
Session session =
```

```
HibernateUtil  
.getSessionFactory()  
.openSession();
```

```
Student student =
```

```
session.get(  
Student.class,  
1  
);
```

```
System.out.println(  
student.getName()  
);
```

```
session.close();
```

OUTPUT

Swati



FLOW

ID
↓
session.get()
↓
Database
↓
Student Object



7. WHAT IF RECORD NOT FOUND?

Example

Student student=

```
session.get(  
Student.class,  
100  
);
```

Output

null

Always check:

```
if(student!=null){  
  
}
```



8. UPDATE OPERATION

Database

id	name	course
1	Swati	Java

Need:

Java → Spring Boot

STEP 1

Fetch Record

Student s=

```
session.get(
Student.class,
1
);
```

STEP 2

Modify Object

```
s.setCourse(
"Spring Boot"
);
```

STEP 3

Update

```
session.merge(s);
```

STEP 4

Commit

```
tx.commit();
```

COMPLETE UPDATE PROGRAM

Session session=

```
HibernateUtil  
.getSessionFactory()  
.openSession();
```

Transaction tx=

```
session.beginTransaction();
```

Student s=

```
session.get(  
Student.class,  
1  
);
```

```
s.setCourse(  
"Spring Boot"  
);
```

```
session.merge(s);
```

```
tx.commit();
```

```
session.close();
```



FLOW

Database Record



Fetch Object



Modify Object



merge()



UPDATE SQL

GENERATED SQL

update student

set course='Spring Boot'

where id=1;



9. DELETE OPERATION

Need to remove:

Student ID = 1

STEP 1

Fetch Record

Student s=

```
session.get(
Student.class,
1
);
```

STEP 2

Delete

```
session.remove(s);
```

STEP 3

Commit

```
tx.commit();
```

COMPLETE DELETE PROGRAM

Session session=

```
HibernateUtil  
.getSessionFactory()  
.openSession();
```

Transaction tx=

```
session.beginTransaction();
```

Student s=

```
session.get(  
Student.class,  
1  
);
```

```
session.remove(s);
```

```
tx.commit();
```

```
session.close();
```

GENERATED SQL

delete from student

where id=1;



FLOW

Fetch Record



remove()



DELETE SQL



Database



10. TRANSACTION MANAGEMENT

Very Important.

Why Transaction?

Suppose:

Insert Student

Insert Fees

Insert Course

If error occurs:

Data becomes inconsistent.

Transaction ensures:

All Success

OR

All Fail

Begin Transaction

Transaction tx=

```
session.beginTransaction();
```

Commit

```
tx.commit();
```

Rollback

```
tx.rollback();
```

Example

```
try{  
  
tx.commit();  
  
}  
catch(Exception e){  
  
tx.rollback();  
  
}
```



11. COMPLETE CRUD FLOW

CREATE

↓

`persist()`

READ

↓

`get()`

UPDATE

↓

`merge()`

DELETE

↓

`remove()`

MEMORY TABLE

Operation	Method
-----------	--------

Create	<code>persist()</code>
--------	------------------------

Read	<code>get()</code>
------	--------------------

Update	<code>merge()</code>
--------	----------------------

Delete	<code>remove()</code>
--------	-----------------------



12. REAL STUDENT MANAGEMENT SYSTEM

Add Student

`persist()`

View Student

`get()`

Update Student

merge()

Delete Student

remove()



COMPLETE SYSTEM

Add Student



Database



View Student



Update Student



Delete Student



13. INTERVIEW QUESTIONS

What is CRUD?

Create Read Update Delete.

Which method inserts data?

persist()

Which method fetches data?

get()

Which method updates data?

merge()

Which method deletes data?

remove()

Why Transaction is required?

To maintain database consistency.

Difference between get() and persist()?

get()	persist()
Fetch Data	Insert Data

COMMON STUDENT MISTAKES

Mistake 1

Forget commit.

```
tx.commit();
```

Result:

Data not saved.

Mistake 2

Delete without fetching.

Wrong:

```
session.remove(1);
```

Correct:

```
Student s=  
session.get(  
Student.class,  
1  
);
```

```
session.remove(s);
```

Mistake 3

Update without transaction.

Mistake 4

Using closed session.



DAY 4 ASSIGNMENT

Theory

1. What is CRUD?
 2. Explain persist().
 3. Explain get().
 4. Explain merge().
 5. Explain remove().
 6. Why Transaction is required?
 7. Difference between get() and persist().
-

Practical

Task 1

Insert 5 students.

Swati
Rahul
Amit
Neha
Priya

Task 2

Fetch Student ID = 3.

Display details.

Task 3

Update:

Java → Spring Boot

Task 4

Delete Student ID = 2.

Task 5

Create Menu Driven Program.

1 Add Student

2 View Student

3 Update Student

4 Delete Student

5 Exit



WEEK 5 → DAY 5 DEEP

COMPLETE STUDENT REGISTRATION SYSTEM USING HIBERNATE



DAY 5 OBJECTIVE

By the end of this session students should understand:

- ✓ Real Hibernate Project Structure
 - ✓ Student Registration System
 - ✓ DAO Layer
 - ✓ Hibernate Integration
 - ✓ Insert Student
 - ✓ View Student
 - ✓ MVC Architecture
 - ✓ Industry Project Flow
 - ✓ End-to-End Hibernate Application
-



1. WHAT ARE WE BUILDING TODAY?

A Complete Student Registration System.

FEATURES

1. Add Student
2. View Student

3. Update Student

4. Delete Student

5. Exit

APPLICATION FLOW

User



Main Class



StudentDAO



Hibernate



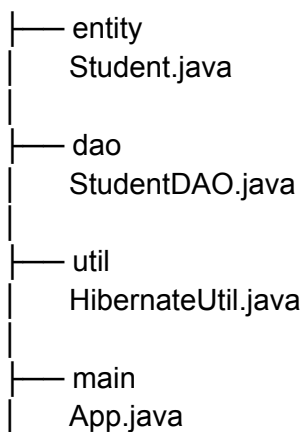
MySQL



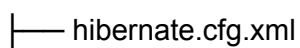
2. PROJECT STRUCTURE

HibernateStudentProject

src/main/java



src/main/resources





3. DATABASE SETUP

Create Database

```
create database studentdb;
```

Use Database

```
use studentdb;
```

No table creation needed.

Hibernate creates automatically.



4. ENTITY CLASS

Student.java

THEORY

Entity Class represents database table.

Every object becomes one row.

Example

Student Object



Student Table Row

CODE

```
package entity;

import jakarta.persistence.*;

@Entity
@Table(name="student")
public class Student {

    @Id
    @GeneratedValue(strategy=
    GenerationType.IDENTITY)

    private int id;

    private String name;

    private String course;

    private double fees;

    public Student() {

    }

    public Student(
    String name,
    String course,
    double fees
    ){
    this.name=name;
    this.course=course;
    this.fees=fees;
    }

    public int getId() {
    return id;
    }

    public void setId(int id) {
    this.id=id;
    }

    public String getName() {
    return name;
    }
}
```

```
public void setName(String name) {
    this.name=name;
}

public String getCourse() {
    return course;
}

public void setCourse(String course) {
    this.course=course;
}

public double getFees() {
    return fees;
}

public void setFees(double fees) {
    this.fees=fees;
}

}
```



5. hibernate.cfg.xml

THEORY

Stores database configuration.

Without this file Hibernate cannot connect.

```
<?xml version='1.0'?>
```

```
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
```

```
<hibernate-configuration>
```

```
<session-factory>
```

```
<property name="hibernate.connection.driver_class">
com.mysql.cj.jdbc.Driver
</property>
```

```
<property name="hibernate.connection.url">
jdbc:mysql://localhost:3306/studentdb
</property>
```

```
<property name="hibernate.connection.username">
root
</property>
```

```
<property name="hibernate.connection.password">
root
</property>
```

```
<property name="hibernate.dialect">
org.hibernate.dialect.MySQLDialect
</property>
```

```
<property name="hibernate.hbm2ddl.auto">
update
</property>
```

```
<property name="hibernate.show_sql">
true
</property>
```

```
<mapping class="entity.Student"/>
```

```
</session-factory>
```

```
</hibernate-configuration>
```



6. HIBERNATE UTIL CLASS

THEORY

SessionFactory is expensive.

Create only once.

Reuse everywhere.

```
package util;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {

    private static SessionFactory
    sessionFactory;

    static {

        Configuration cfg =
        new Configuration();

        cfg.configure();

        sessionFactory =
        cfg.buildSessionFactory();

    }

    public static SessionFactory
    getSessionFactory(){

        return sessionFactory;

    }

}
```



7. DAO CONCEPT

Very Important.

WHAT IS DAO?

DAO = Data Access Object

PURPOSE

Separate database code from business code.

Wrong:

App.java

Database Logic

Correct:

App.java



StudentDAO



Database

BENEFITS

- ✓ Cleaner Code
 - ✓ Easy Maintenance
 - ✓ Industry Standard
-



8. STUDENT DAO

StudentDAO.java

INSERT METHOD

```
public void addStudent(  
    Student student  
)  
{  
  
    Session session=  
        HibernateUtil  
            .getSessionFactory()  
            .openSession();  
  
    Transaction tx=  
        session.beginTransaction();  
  
    session.persist(student);  
  
    tx.commit();  
  
    session.close();  
}
```

THEORY

Student Object
↓
persist()
↓
Database



9. VIEW STUDENT

THEORY

Need to fetch student by ID.

```
public Student getStudent(  
    int id  
)  
{  
  
    Session session=  
        HibernateUtil  
            .getSessionFactory()  
            .openSession();  
  
    Student student=  
  
        session.get(  
            Student.class,  
            id  
        );  
  
    session.close();  
  
    return student;  
}
```

FLOW

```
ID  
↓  
get()  
↓  
Database  
↓  
Student Object
```



10. UPDATE STUDENT

```
public void updateStudent(  
    Student student  
)  
{  
  
    Session session=  
        HibernateUtil
```

```
.getSessionFactory()
.openSession();

Transaction tx=
session.beginTransaction();

session.merge(student);

tx.commit();

session.close();

}
```

FLOW

Object
↓
merge()
↓
UPDATE SQL



11. DELETE STUDENT

```
public void deleteStudent(
int id
){
```

```
Session session=
HibernateUtil
.getSessionFactory()
.openSession();
```

```
Transaction tx=
session.beginTransaction();
```

```
Student student=
```

```
session.get(
Student.class,
```

```
id
);

if(student!=null){

session.remove(student);

}

tx.commit();

session.close();

}
```

FLOW

```
ID
↓
get()
↓
remove()
↓
DELETE SQL
```



12. COMPLETE DAO CLASS

StudentDAO

└─ addStudent()

└─ getStudent()

└─ updateStudent()

└─ deleteStudent()



13. MAIN APPLICATION

App.java

THEORY

This is menu-driven program.

User selects operation.

DAO performs database task.

```
package main;

import java.util.Scanner;

import dao.StudentDAO;
import entity.Student;

public class App {

    public static void main(
        String[] args
    ){

        Scanner sc=
            new Scanner(System.in);

        StudentDAO dao=
            new StudentDAO();

        while(true){

            System.out.println(
                "\n1 Add Student"
            );

            System.out.println(
                "\n2 View Student"
            );

            System.out.println(
                "\n3 Update Student"
            );

            System.out.println(
```

```
"4 Delete Student"  
);
```

```
System.out.println(  
"5 Exit"  
);
```

```
int choice=  
sc.nextInt();
```

```
switch(choice){
```

```
case 1:
```

```
System.out.println(  
"Enter Name:"  
);
```

```
String name=  
sc.next();
```

```
System.out.println(  
"Enter Course:"  
);
```

```
String course=  
sc.next();
```

```
System.out.println(  
"Enter Fees:"  
);
```

```
double fees=  
sc.nextDouble();
```

```
Student student=  
new Student(  
name,  
course,  
fees  
);
```

```
dao.addStudent(  
student  
);
```

```
System.out.println(  
"Student Added"
```

```
);  
  
break;  
  
case 5:  
  
System.exit(0);  
  
}  
  
}  
  
}  
  
}
```



14. COMPLETE PROJECT FLOW

Add Student

```
graph TD  
    A[User Input] --> B[Student Object]  
    B --> C[DAO]  
    C --> D[Hibernate]  
    D --> E[Database]
```

View Student

```
graph TD  
    A[ID] --> B[DAO]  
    B --> C[Hibernate]  
    C --> D[Database]  
    D --> E[Student Object]
```

Update Student

Modified Object

↓
DAO
↓
merge()
↓
Database

Delete Student

ID
↓
DAO
↓
remove()
↓
Database



15. REAL INDUSTRY ARCHITECTURE

Presentation Layer

↓
Controller Layer
↓
Service Layer
↓
DAO Layer
↓
Hibernate
↓
Database

OUR PROJECT

App.java

↓
StudentDAO
↓
Hibernate
↓
MySQL



16. COMPLETE WEEK 5 REVISION

DAY 1

What is Hibernate

ORM

JDBC Problems

DAY 2

Architecture

Configuration

SessionFactory

Session

Transaction

DAY 3

Entity Class

@Entity

@Table

@Id

@GeneratedValue

@Column

DAY 4

CRUD

persist()

get()

merge()

remove()

DAY 5

Student Registration Project

DAO Layer

Hibernate Integration

Industry Flow



INTERVIEW QUESTIONS

What is DAO?

Data Access Object.

Used to separate database logic.

Why Hibernate Util Class?

To create SessionFactory only once.

Which method inserts data?

`persist()`

Which method updates data?

`merge()`

Which method deletes data?

`remove()`

Why Entity Class is needed?

To map Java object with database table.

Why Transaction is required?

To save changes permanently.



DAY 5 ASSIGNMENT

Theory

1. What is DAO?
 2. Explain Student Registration Flow.
 3. Explain HibernateUtil.
 4. Explain Entity Class.
 5. Explain CRUD using Hibernate.
 6. Explain complete Hibernate Architecture.
 7. Difference between JDBC and Hibernate.
-

Practical Assignment

Project 1

Student Registration System

Fields:

Id
Name
Course
Fees

Operations:

Add Student

View Student

Update Student

Delete Student

Project 2

Employee Management System

Fields:

Id
Name
Department
Salary

Perform complete CRUD.
